



KNOWLEDGE GRAPHS

From Data to Intelligent Applications

Data → Information → Knowledge → Wisdom → Intelligent Applications

Six-Week Roadmap

How the course builds from raw data up to graph-powered intelligent applications

01

Knowledge Representation with Graphs

DIKW pyramid, meaning, triples, the Semantic Web, Linked Data

02

Basic Knowledge Graph Infrastructure

URIs, RDF, Turtle, RDFS, inference, reification

03

Querying Knowledge Graphs with SPARQL

Query anatomy, DBpedia vs Wikidata, advanced features, SHACL

04

Ontologies and Logic

Why RDFS falls short, Description Logics, OWL

05

Ontological Engineering for Smarter KGs

Rules, the Ontology 101 recipe, evaluation, alignment

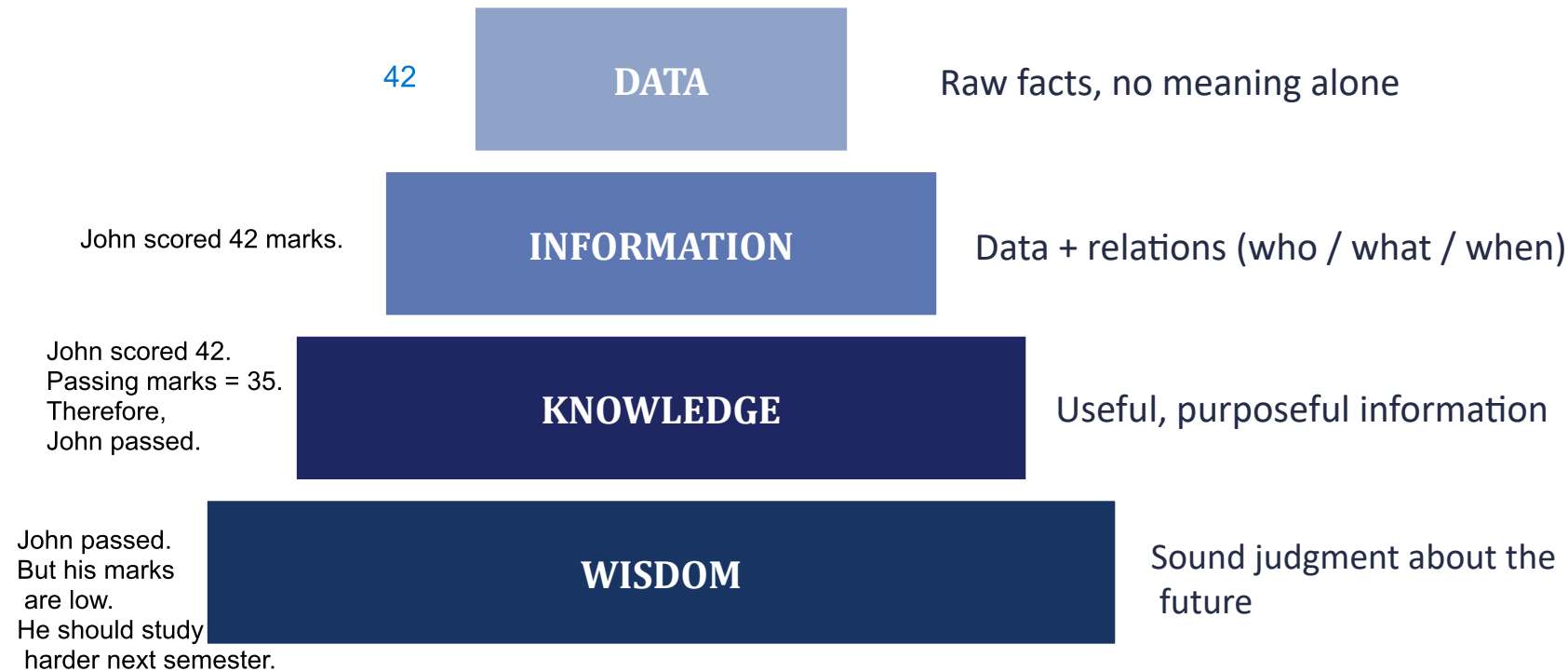
06

Intelligent Applications: KGs + Deep Learning

Graph theory, embeddings, completion, KGs meeting LLMs

From Data to Knowledge — the DIKW Pyramid

Take the number '42': on its own it's meaningless. The DIKW pyramid tracks the journey it takes before it becomes genuinely useful.



THINK OF IT LIKE THIS

Data is individual LEGO bricks. Information is bricks sorted by colour and size. Knowledge is the instruction booklet. Wisdom is knowing which model to build for the situation you're in.

JTB — Justified True Belief

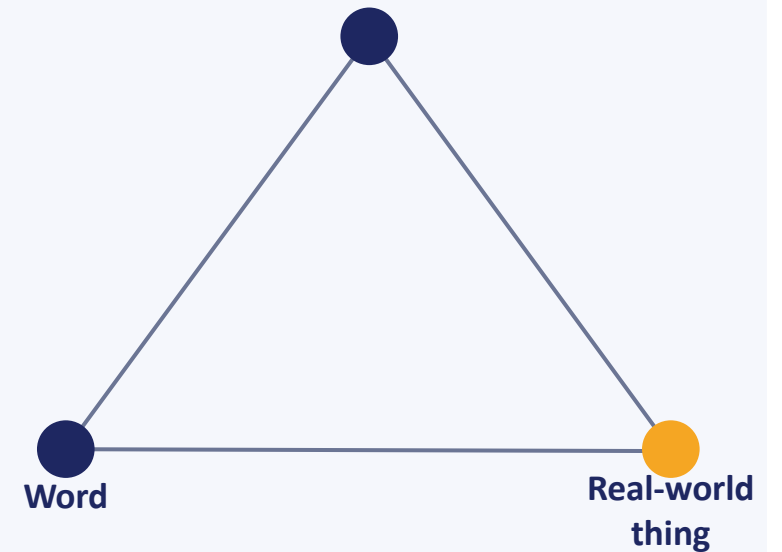
You only truly 'know' something if it's actually true, you believe it, and you have good justification. Miss any one and it's just a lucky guess or a false belief.

The Art of Understanding — 5 Ingredients of Meaning

Understanding a sentence takes more than knowing the words. Five ingredients work together:

- 1 Syntax** Is the grammar / structure correct?
☒ "The boy eats an apple."
☒ "Apple eats boy the."
- 2 Semantics** What does that structure actually mean?
"The cat is sleeping."
Means the cat is resting
- 3 Context** The surrounding situation can flip the meaning — 'Jaguar'
means something different in a jungle than in a car showroom
- 4 Pragmatics** What did the speaker actually intend — literal or
sarcastic?
Ex- "What lovely weather!"
- 5 Experience** World knowledge needed to catch culture-specific
references
Ex - "He scored a century."

THE SEMIOTIC TRIANGLE Concept



Communication only works because both people are pointing at the same concept in their heads.

Example:

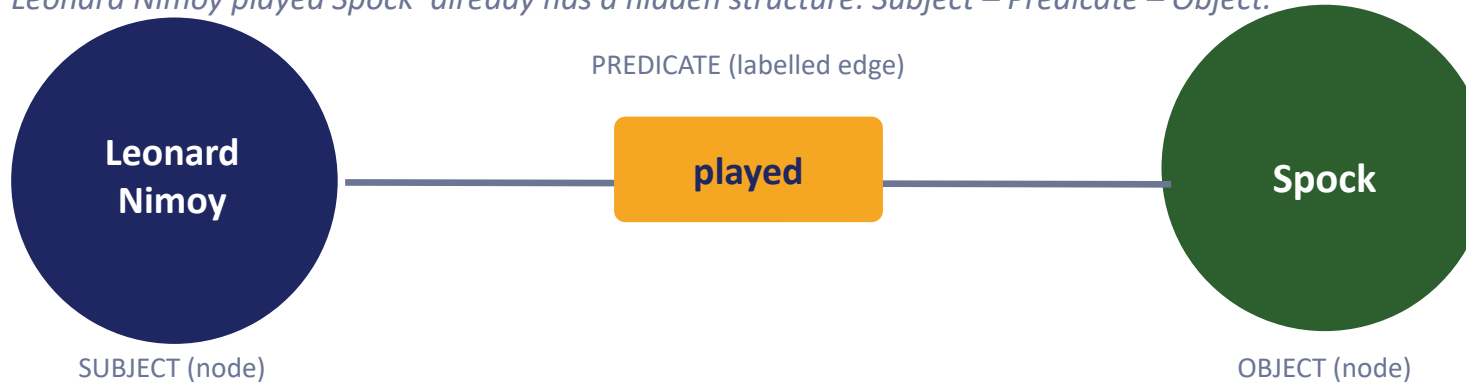
Word: "Dog"

Concept: Your mental idea of a dog.

Real object: An actual dog in the world.

Graphs and Triples — the Building Block of Everything

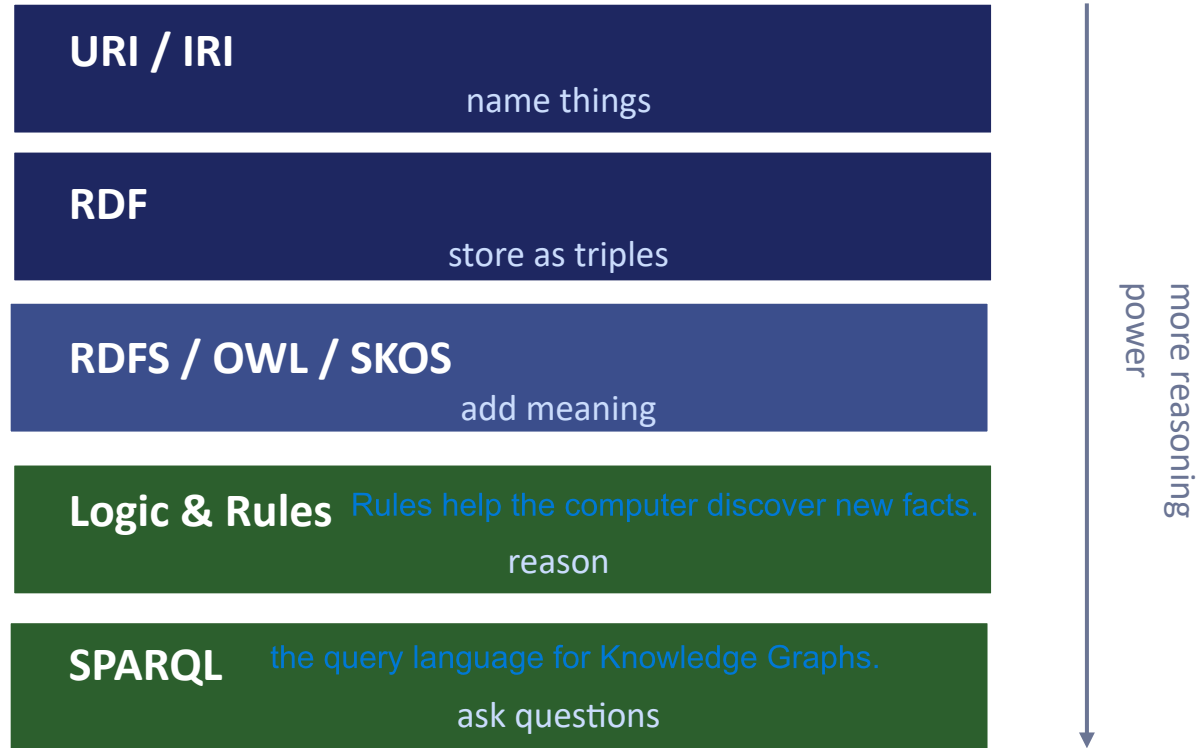
A sentence like 'Leonard Nimoy played Spock' already has a hidden structure: Subject – Predicate – Object.



Formally, a graph is $G = (V, L, E)$ — V = nodes, L = edge labels, E = (subject, label, object) connections.

- **Entity** — Something that exists independently (a node) Ex - India
- **Relation** — How two entities connect (an edge) India — capital —► New Delhi
- **Literal** — A plain data value with no independent existence — dates, numbers — drawn as a box, not a circle Ex- Age = 21
- **Class** — A group of individuals sharing properties, e.g. 'Person'
- **Property graph** — Triples plus extra attributes on nodes/edges — richer, but still doesn't pin down what a property truly means

The Semantic Web Stack & Linked Data



Layer cake: URIs give everything a name tag, RDF stores the triples, RDFS/OWL/SKOS add meaning, Logic & Rules let you reason, SPARQL lets you ask questions of the whole cake.

Linked Data — a Web of Facts

- Use URIs as names for things
URI (Uniform Resource Identifier) is a unique identifier for anything.
- Use HTTP URIs so those names are look-up-able
The URI should use HTTP or HTTPS so anyone can open it in a browser.
- Return useful RDF/SPARQL data when someone looks one up
- Link to other people's URIs, not just your own

5-STAR OPEN DATA

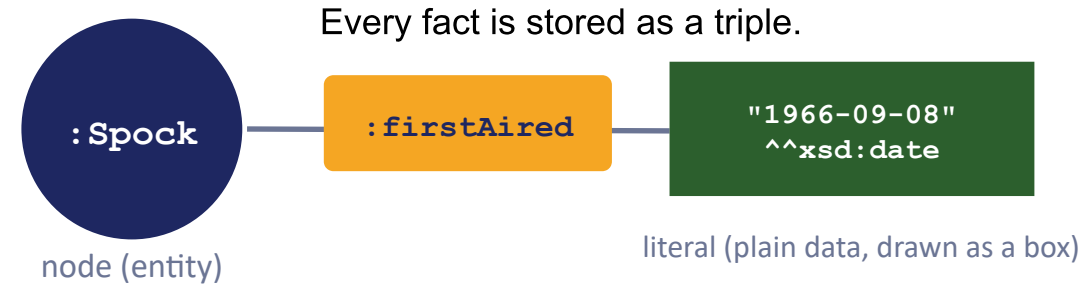
Rates how linkable your data is: one star for any open format at all, up to five stars for data that's in RDF and linked to everyone else's data. This naturally satisfies most of FAIR — Findable, Accessible, Interoperable, Reusable.

Giving Things Names, and Writing Facts Down

Naming Things on the Web

- **URI / IRI** — A unique identifier for any resource; IRI extends URI to support Unicode
Souvik
↓
<https://example.org/person/souvik>
- **Structure** — Built from a scheme, a domain/host, a path, and an optional fragment
Ex: <https://www.wikidata.org/entity/Q937>
- **Designator vs. Designatum** — A description/URI is not the real-world object itself — HTTP content negotiation bridges the two The name or identifier. and The real object.
- **Disambiguation** — Two people named 'Paris' get solved with language-independent numeric IDs — Wikidata's Q-numbers are the classic example
[Disambiguation means removing confusion.]

RDF — Writing Facts Down



- **Literals** — Come in three flavours: plain ("Spock"), typed (^^xsd:date), or language-tagged (@en)
- **Blank node** — A way to say 'something exists' without giving it a proper name — can't be referenced from outside the graph
- **Serialisations** — N-Triples (verbose), Turtle (compact, used throughout this course), RDF/XML, JSON-LD, N-Quads (adds a graph name)
[Different ways of writing the same RDF data.]

Turtle Shorthand & RDFS Meaning

Turtle — Shorthand for Writing RDF by Hand

@**prefix** /
@**base**

Shorten long, repetitive URIs into short tags

;

Same subject, next property
; = Same Person

,

Same subject AND property, just another value
, = Same person + same property

[]

An anonymous blank node

_:id

A named, reusable blank node — handy for relations involving more than two things

Containers, Collections & RDF Datasets

Containers (open-ended): `rdf:Bag`, `rdf:Seq`, `rdf:Alt`. Collections (closed lists): built from `rdf:first/rest/nil`, Turtle shortcut `()`. RDF Dataset: a default graph plus named graphs — tracks where a fact came from.

- Turtle is a simple and readable format for writing RDF.

RDF = English paragraph

Turtle = Short notes

RDFS — Giving Triples Actual Meaning

- **rdf:type** — Links an instance to its class — e.g. ‘:Spock `rdf:type :Vulcan`’ Ex- Souvik is a Student.
- **rdfs:domain / range** — Restrict what class the subject/object of a property must belong to Rahul studiesAt DRBCREC means he is a student
- **rdfs:subClassOf / subPropertyOf** — Build hierarchies — inheritance follows automatically Souvik is a Person.
- **Annotation only** — `rdfs:label`, `rdfs:comment`, `rdfs:seeAlso`, `rdfs:isDefinedBy` — no reasoning power

T-BOX vs. A-BOX

T-Box is the rulebook / schema (‘a Vulcan is a kind of Humanoid’). A-Box is the actual roster of players (‘Spock is a Vulcan’). This same split reappears in ontologies.

Contains the rules.
Student is a Person

Contains the actual data.
Souvik is a Student

Logical Inference, Reification & RDF-star

Entailment is the logical consequence of what you already know; inference is the process of actually computing it.

Inference means finding a new fact from existing facts.

Rule 1 — subClassOf

If I is a C1, and C1 is a subclass of C2, then I is also a C2

Rule 2 — domain/range

If property P connects i1 to i2, and P's domain is C1 / range is C2, then i1 is a C1 and i2 is a C2

Rule 3 — subPropertyOf

If P1 is a sub-property of P2, then anything connected by P1 is also connected by P2

Rahul hasFather Amit
means Rahul has parents

Reasoning comes in three flavours — deductive: rules to conclusion (this course's focus); inductive: spotting patterns (the ML approach); abductive: picking the best explanation.

Reification & RDF-star

Sometimes you need to say something about a fact itself — who claimed it, or when. Classic reification wraps the triple in an `rdf:Statement` (correct but verbose). RDF-star's `<<S P O>>` lets a whole triple act as subject or object of another triple — compact, great for provenance.

RDFa & RDFLib

RDFa embeds structured data in HTML (vocab, typeof, property, resource, content, datatype). In RDFLib: `add()` appends a triple (duplicates allowed), `set()` overwrites — like appending to a list vs. assigning a single value.

SPARQL Basics — SQL for Graphs

A SPARQL endpoint is a server hosting a triple store that you can query over plain HTTP — a database, but for graphs instead of tables.

```
SELECT ?person ?job
WHERE {
  ?person :hasJob ?job .
  FILTER(?job = "Actor")
}
```

← columns you want back

← graph pattern to match

← triple pattern with variables

← narrow the results

- variable binds to whatever matches in the graph; SELECT lists columns, just like SQL
- Graph pattern matching: triple patterns match anywhere — unlike SQL's fixed rows and tables
- Modifiers: ORDER BY (ASC/DESC), LIMIT, OFFSET
- FILTER(...) narrows results — comparisons, bound(), type/language checks

Life Beyond SELECT

- **ASK** — Just true/false
- **DESCRIBE** — Every triple about the matched resources
- **CONSTRUCT** — Builds brand-new triples from a template
- **Results** — Can come back as CSV, XML, or JSON

Property Paths — one-step traversal

	alternatives	^	inverse direction
+	one-or-more	*	zero-or-more
!	negated path	/	sequence

Two Big Public Knowledge Graphs

DBpedia

DBpedia extracts information automatically from Wikipedia infoboxes.

- Auto-extracted from Wikipedia infoboxes
- Has both a clean ontology-based vocabulary (dbo:) and a messier raw one (dbp:)
- Imports Wikipedia's category system via SKOS broader

Wikidata

Wikidata is a community-edited Knowledge Graph maintained by volunteers. Unlike DBpedia, people edit the data directly.

- Community-edited; Q-numbers for entities, P-numbers for properties
- Every statement can carry references (sources) and qualifiers (e.g. start/end date)
- Runs on MediaWiki + MariaDB, with RDF generated as a synced layer — richer in content, weaker formal class logic than a strict ontology

SERVICE (Wikidata's label service) turns cryptic Q-numbers into human-readable labels automatically — useful for federated queries that pull from several knowledge graphs at once, usually linked via owl:sameAs.

Advanced Query Features & Quality Assurance

Advanced SPARQL Features

- **REGEX**(?var, "pattern", "i") — Pattern matching, case-insensitive with the "i" flag
- **OPTIONAL** { } — Keeps a row even when that part of the pattern doesn't match
- **UNION** — Logical OR between two graph patterns
- **Negation** — FILTER NOT EXISTS, MINUS, !BOUND(?var)
- **BIND** — Creates a new variable from a function, e.g. YEAR(?date)
- **Aggregates** — COUNT, SUM, AVG, MIN, MAX, SAMPLE, GROUP_CONCAT — with GROUP BY
- **Subselect** — A nested SELECT, exactly like a SQL subquery

Open-World Assumption

The Semantic Web assumes open-world: if something isn't stated, it's unknown, not false — which keeps models endlessly extensible. OWL never assumes two differently-named things are automatically distinct (no unique name assumption) — sameness/difference must be stated explicitly with owl:sameAs / owl:differentFrom.

SHACL — Shapes Constraint Language

Flips the open-world assumption around: validates a graph against closed-world-style rules, like a strict data-quality checklist. Key building blocks: sh:NodeShape, sh:targetClass, sh:property, sh:minCount / sh:maxCount, and datatype constraints — shapes can even inherit from other shapes.

What Is an Ontology, and Why Isn't RDFS Enough?

Ontology (Greek: 'science of being')

Gruber's definition: an ontology is “an explicit, formal specification of a shared conceptualization.” Built from four pieces: **Classes**, **Relations**, **Constraints**, **Axioms**. Same T-Box / A-Box split as before.

An Ontology is a set of rules that describes how concepts in a domain are related.

Constraints=Rules.
Axioms=Logical truths.

Why Plain RDFS Isn't Enough

RDFS can't express: restrictions scoped to one subclass, non-overlapping classes, union/intersection of classes, exact cardinality, transitivity, functional (unique) properties, inverse properties — and, the real blocker, negation. Without negation there's no way to formally detect a genuine contradiction.

A Fast Logic Recap

Propositional Logic: atoms + AND/OR/NOT/IF/IFF, truth tables. First-Order Logic: adds predicates, quantifiers, functions — expressive but only semi-decidable.

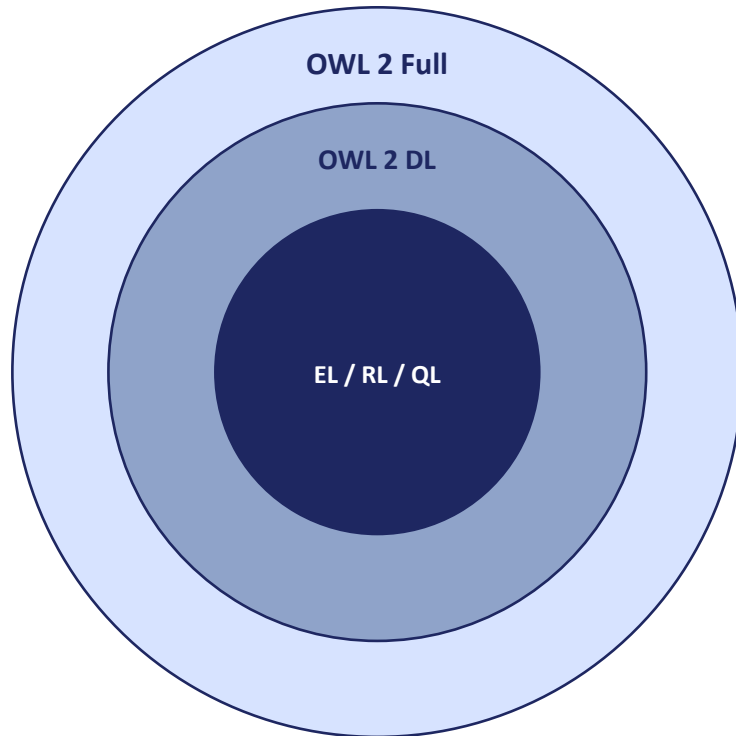
Description Logics — the Practical Middle Ground

A simplified version of mathematics that computers can understand.

A carefully chosen, decidable slice of First-Order Logic — expressive enough to be useful, restrained enough that a computer is guaranteed to finish reasoning. DL talks about Concepts (classes), Roles (properties), and Individuals.

- **ALC** The smallest deductively-complete DL: conjunction, disjunction, negation, existential & universal restriction
- **T-Box / A-Box** C subsumes D, or C is equivalent to D / C(a), R(a,b)
- **Extensions** S = transitive roles, H = role hierarchy, O = nominals, I = inverse roles, Q = qualified cardinality, D = datatypes
- **OWL 2 DL** Built on a description logic called SHOIN(D)

OWL — the Web Ontology Language



smaller circle = less expressive, but faster / decidable reasoning

Profiles nest inside each other. This course sticks to OWL 2 DL, because it stays computable.

OWL is simply the language used to write the ontology.

- **owl:Thing / owl:Nothing** — Everything (top) / the empty class (bottom)

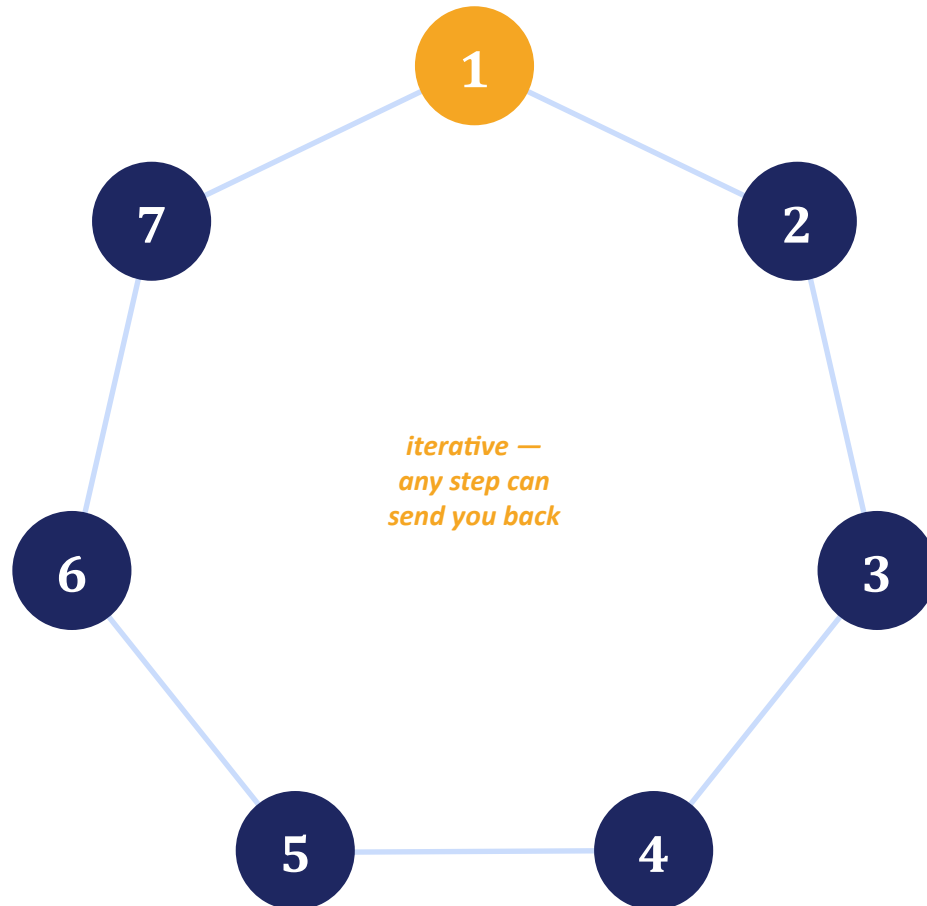
owl:Thing
The biggest class.
Everything belongs here.

Thing
├── Person
├── Animal
├── Car
└── Book

- **owl:ObjectProperty / DatatypeProperty** — Points to another entity / a plain literal
Object Property
Connects two objects.
Datatype Property
Connects object with data.
- **disjointWith, equivalentClass, sameAs, differentFrom** — All must be stated explicitly — the web assumes an open world
- **Class builders** — oneOf (closed list), intersectionOf, unionOf, complementOf
- **Restrictions** — hasValue (one value), allValuesFrom (for-all), someValuesFrom (there-exists), cardinality / min / max
- **Property types** — inverseOf, equivalentProperty, TransitiveProperty (plus Functional / Symmetric)

Designing an Ontology — the ‘Ontology 101’ Recipe

Seven steps, but never a straight line — expect to loop back constantly.



- 1. Determine scope, domain & purpose — write competency questions with domain experts
- 2. Consider reusing an existing ontology — saves effort, comes with validated tooling
- 3. Enumerate the terms relevant to the domain
- 4. Define classes and their hierarchy — top-down, bottom-up, or middle-out
- 5. Define the properties that connect classes
- 6. Define property constraints and facets
- 7. Create the actual instances

Evaluating, Aligning & Building Ontologies

Evaluating

- Verification: is it technically correct?
- Validation: does its meaning match intended reality?
- Judged on: accuracy, adaptability, clarity, completeness, efficiency, conciseness, consistency, org. fitness
- Accuracy is most often measured with Precision & Recall
Precision: how many were correct?
Recall: how many did AI find?

Aligning & Learning

- Alignment: correspondences between two ontologies, tagged with a relation type + confidence score
- Mismatches: syntactic, terminological, conceptual/semantic, semiotic/pragmatic
- Ontology learning: deriving an ontology from text, Linked Data, or crowdsourcing
- Layer cake (easy→hard): terms → synonyms → concept description → hierarchy → relations → relation hierarchy → axioms

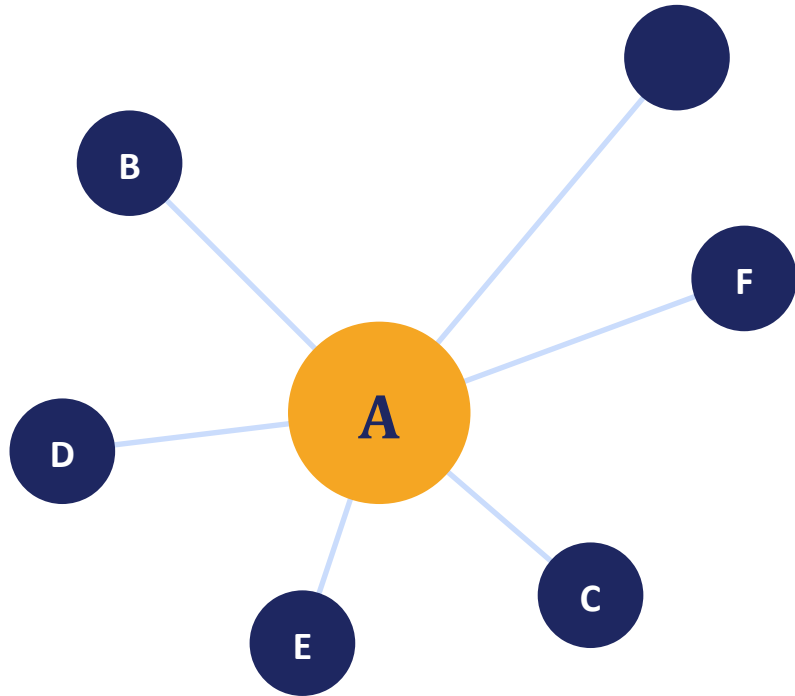
Building in Practice

- From text: entity recognition → linking/resolution → relation extraction → property linking
- LLM-based extraction spots entities/dates well but produces imprecise links — human verification still needed
- From structured data: define a mapping, then materialize or virtualize
- R2RML maps DB tables to RDF via a triples map — one set of triples per row

An ontology is not reality — it's a purpose-built model of it. Always ask first: do you even need one? Name things unambiguously, and don't confuse subsumption with instantiation, part-whole, disjunction, or constitution.

Graph Theory Basics — Centrality

Formally $G = (V, E)$ with directed edges as ordered pairs — plus self-loops, multigraphs, and edge/vertex-labelled graphs.



Node A has the highest degree — a hub in the graph

- **Degree** — How many edges touch a vertex — split into in-degree and out-degree for directed graphs
- **Strongly connected** — A path exists between every single pair of vertices
- **Centrality** — Measures how important a node is: degree, eigenvector, Katz centrality, and the famous PageRank
- **Graph stats** — Size, eccentricity (furthest distance from one node), diameter (largest eccentricity), radius (smallest)
- **Watch out for** — Trivially high-degree nodes like owl:Thing — exclude them before computing importance, or they'll skew everything

Knowledge Graph Embeddings

Word2vec's core idea — a word's meaning is defined by the company it keeps — extends naturally to graph entities.

King



Man

Queen



Woman

king – man + woman $\approx$ queen (same idea applies to KG entities)

Good embeddings capture patterns like symmetry, anti-symmetry, inversion, composition, transitivity, and hierarchy.

The Embedding 'Construction Kit'

- **Scoring function** — Rates how plausible a triple is
- **Loss function** — Penalises the model when a true triple scores lower than a fake one
- **TransE (baseline)** — Subject-vector + relation-vector should land close to object-vector (TransH/R/D are variants)
- **RotatE** — Represents relations as rotations in complex-number space
- **Other families** — Factorization (RESICAL, DistMult, ComplEx), convolution (ConvE, ConvKB), walk-based (node2vec, RDF2Vec, DeepWalk)

Knowledge Graph Completion, & Meeting LLMs

Knowledge Graph Completion

KGs live under the open-world assumption, so they're always incomplete. Refinement covers completion (adding missing facts) and error detection (catching wrong ones).

- **Sub-tasks** — tail prediction (S,P,?), head prediction (?,P,O), relation prediction (S,?,O), triple classification, entity/type classification
- **TransE in practice** — Take the subject vector, add the relation vector, find the nearest entity vector — that's the predicted object
- **Transductive vs. Inductive** — Only known entities vs. handling new, unseen entities (fully- and semi-inductive variants exist too)
- **Also used for** — Identity prediction (duplicates), fact-checking, and correcting the KG itself

Knowledge Graphs Meet LLMs

LLMs hallucinate with total confidence — Galactica famously fabricated plausible-sounding but wrong facts and citations. They also struggle with long-tail topics, lack real-world grounding, and inherit training-data biases.

HYBRID AI

Deep learning generalises poorly but learns patterns well; symbolic AI (a KG) abstracts cleanly but doesn't scale. Together, each covers the other's blind spot.

- **Explainable AI** A path through the KG can explain why an LLM gave a particular answer
- **Fact-checking** Map an LLM's claims onto the KG and check consistency
- **Bottlenecks** Coverage (a KG can't encode everything), reasoning cost, and provenance/trust — still needs human judgment

Semantic Search & Graph-Powered Recommenders

Semantic Search

Traditional keyword search (bag-of-words) has no real understanding — it struggles with synonyms, paraphrasing, and implied meaning. Weaving a KG into the pipeline unlocks query refinement, cross-referencing, fuzzy search, and reasoning over implicit facts.

- 1 Exact entity matching
- 2 Similarity-based matching (shared property values)
- 3 Relationship-based matching (connected entities)

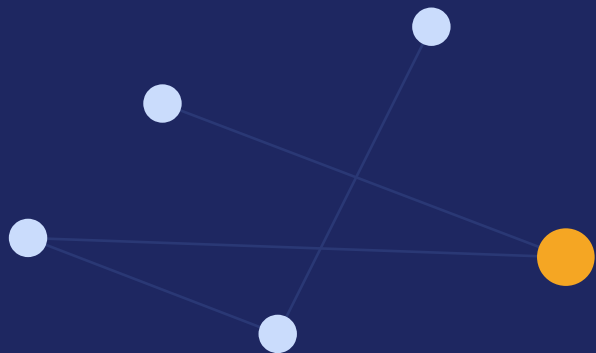
Easiest → hardest, left to right above.

Exploratory Search & Recommenders

- **Retrieval vs. Exploration** — You know exactly what you want, vs. a fuzzy goal — discovering related things as you go
- **Similarity vs. Relatedness** — Shares property-values (same author/genre) vs. connected but structurally different (a film adaptation of a book)
- **Multi-hop graph walks** — book → author → their other books → related author → their books — diverse and explainable, since the path IS the explanation
- **Best recommenders** — Combine content similarity/relatedness with the user's own context and history

Course Recap — One Line Per Week

- 1 Data becomes knowledge; graphs and triples; what a Knowledge Graph is; the Semantic Web and Linked Data
- 2 RDF (triples, Turtle) + RDFS (schema and meaning) + inference + reification
- 3 SPARQL — querying, filtering, federating, and validating (SHACL) knowledge graphs
- 4 Ontologies formalised with logic — Description Logics leading up to OWL
- 5 Designing, evaluating, aligning, and building ontologies / KGs the right way
- 6 Turning graphs into ML features (embeddings) — completion, search, recommenders, and pairing KGs with LLMs



From Raw Data to Intelligent Applications

Knowledge Graphs give machines a structured, reasoning-ready map of meaning — the missing link between raw data and genuine understanding.

Thank You